U25AI055

Lab-4

1. **Find the middle element of a singly linked list.**
   Example: 11 → 22 → 33 → 44 → 55 → NULL → Middle = 33

```
// 1. Find the middle element of a singly linked list.
#include <stdio.h>
#include <stdlib.h>
struct Node
{
  int data;
  struct Node *next;
};
struct Node *findMiddleElement(struct Node *head);
int main()
{
  int n;
  printf("Enter number of nodes: ");
  scanf("%d", &n);
  struct Node *head, *newNode, *temp;
  head = NULL;
  for (int i = 1; i <= n; i++)
  {
    newNode = (struct Node*)malloc(sizeof(struct Node));
    printf("Enter element %d: ", i);
    scanf("%d", &newNode->data);
    newNode->next = NULL;
    if (head == NULL)
    {
      head = newNode;
      temp = newNode;
    }
    else
    {
      temp->next = newNode;
      temp = newNode;
    }
  }
  printf("Elements of linked list are ");
  struct Node *ptr;
  ptr = head;
```

```
  while (ptr != NULL)
  {
    printf("%d\t", ptr->data);
    ptr = ptr->next;
  }
  printf("\n");
  ptr = findMiddleElement(head);
  printf("Middle element is %d", ptr->data);
  return 0;
}
struct Node *findMiddleElement(struct Node *head)
{
  struct Node *p, *q;
  q = p = head;
  while (p != NULL && p->next != NULL)
  {
    q = q->next;
    p = p->next->next;
  }
  return q;
}
```

Output:

```
Enter number of nodes: 5
Enter element 1: 11
Enter element 2: 22
Enter element 3: 33
Enter element 4: 44
Enter element 5: 55
Elements of linked list are 11  22      33      44      55
Middle element is 33
```

2. **Detect a loop in a singly linked list.**

```
// 2. Detect a loop in a singly linked list.
#include <stdio.h>
#include <stdlib.h>
struct Node
{
  int data;
  struct Node *next;
};
int detectLoop(struct Node *head);
struct Node *detectLoopStart(struct Node *head);
```

```
int main()
{
   struct Node *head, *node1, *node2, *node3;
   node1 = (struct Node*)malloc(sizeof(struct Node));
   node2 = (struct Node*)malloc(sizeof(struct Node));
   node3 = (struct Node*)malloc(sizeof(struct Node));
   node1->data = 1;
   node1->next = node2;
   node2->data = 3;
   node2->next = node3;
   node3->data = 4;
   node3->next = node2;
   head = node1;
   if (detectLoop(head))
   {
      printf("Loop start detected at the node whose data is %d\n",
detectLoopStart(head)->data);
   }
   else
   {
      printf("Loop not detected\n");
   }
   return 0;
}
int detectLoop(struct Node *head)
{
   struct Node *slow, *fast;
   slow = fast = head;
   while (fast != NULL && fast->next != NULL)
   {
      slow = slow->next;
      fast = fast->next->next;
      if (slow == fast)
      {
         return 1;
      }
   }
   return 0;
}
struct Node *detectLoopStart(struct Node *head)
{
```

```
  struct Node *slow, *fast;
  slow = fast = head;
  while (fast != NULL && fast->next != NULL)
  {
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast)
    {
      slow = head;
      while (slow != fast)
      {
        slow = slow->next;
        fast = fast->next;
      }
      return slow;
    }
  }
  return NULL;
}
```

Output:

```
Loop start detected at the node whose data is 3
```

3. **Remove duplicates from a sorted singly linked list. Example: 2 → 2 → 2 → 5 → 5 → 9 → NULL → 2 → 5 → 9 → NULL**

```
// 3. Remove duplicates from a sorted singly linked list.
#include <iostream>
using namespace std;
struct Node {
  int data;
  Node *next;
};
void remove_element(Node *head, Node *ptr);
Node* remove_duplicates(Node *head);
int main() {
  int n;
  cout << "Enter number of nodes: ";
  cin >> n;
  Node *head, *newNode, *temp;
  head = NULL;
  for (int i = 0; i < n; i++) {
    newNode = new Node;
```

```
        cout << "Enter element " << i + 1 << ": ";
        cin >> newNode->data;
        newNode->next = NULL;
        if (head == NULL) {
            head = newNode;
            temp = newNode;
        } else {
            temp->next = newNode;
            temp = newNode;
        }
    }
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
    cout << endl;
    head = remove_duplicates(head);
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
    return 0;
}
void remove_element(Node *head, Node *ptr) {
    Node *p;
    p = head;
    while (p->next != ptr && p->next != NULL) {
        p = p->next;
    }
    if (p->next == ptr) {
        p->next = ptr->next;
        delete ptr;
    }
}

Node* remove_duplicates(Node *head) {
    Node *ptr;
    ptr = head;
```

```
  while (ptr != NULL && ptr->next != NULL) {
    while (ptr->next != NULL && ptr->data == ptr->next->data) {
      remove_element(head, ptr->next);
    }
    ptr = ptr->next;
  }
  return head;
}
```

Output:

```
Enter number of nodes: 6
Enter element 1: 2
Enter element 2: 2
Enter element 3: 2
Enter element 4: 5
Enter element 5: 5
Enter element 6: 9
2       2       2       5       5       9
2       5       9
```

4. **Clone a linked list with next and random pointers.**

```
// 4. Clone a linked list with next and random pointers.
#include <iostream>
using namespace std;
struct Node {
  int data;
  Node *random;
  Node *next;

};
Node* cloneList(Node *head);
void displayList(Node *head);
int main() {
  Node *head, *node1, *node2, *node3, *node4;
  node1 = new Node;
  node2 = new Node;
  node3 = new Node;
  node4 = new Node;
  head = node1;
```

```
    node1->data = 1;
    node1->next = node2;
    node1->random = node3;
    node2->data = 3;
    node2->next = node3;
    node2->random = node3;
    node3->data = 5;
    node3->next = node4;
    node4->random = NULL;
    node4->data = 9;
    node4->next = NULL;
    node4->random = node3;
    Node *clone = cloneList(head);
    cout << “List cloned” << endl;
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
Node* cloneList(Node *head) {
    Node *clone, *newNode, *temp, *temp1, *temp2, *duplicate, *original;
    clone = NULL;
    original = head;
    while (original != NULL) {
        newNode = new Node;
        newNode->data = original->data;
        newNode->next = NULL;
        if (clone == NULL) {
            clone = newNode;
            temp = newNode;
        } else {
            temp->next = newNode;
            temp = newNode;
        }
        original = original->next;
    }
```

```
    duplicate = clone; original = head;
    while (duplicate != NULL)
    {
      temp1 = original->next;
      original->next = duplicate;
      temp2 = duplicate->next;
      duplicate->next = temp1;
      original = temp1;
      duplicate = temp2;
    }
    original = head;
    while (original != NULL && original->next != NULL) {
      if (original->random != NULL) {
        original->next->random = original->random->next;
      }
      original = original->next->next;
    }
    original = head, duplicate = head->next;
    while (original != NULL) {
      temp = original->next;
      original->next = temp->next;
      if (duplicate->next != NULL) {
        duplicate->next = duplicate->next->next;
      }
      original = original->next;
      duplicate = duplicate->next;
    }
    return clone;
  }
```

Output:

```
List cloned
```

5. **Reverse a singly linked list in groups of size k. Example: 1→2→3→4→5→6→7, k=3 → 3→2→1→6→5→4→7**

```
// 5. Reverse a singly linked list in groups of size k
#include <stdio.h>
#include <stdlib.h>
struct Node
{
  int data;
  struct Node *next;
```

```
};
struct Node *reverseListInGroups(struct Node *head, int k);
void displayList(struct Node *head);
int main()
{
	int n, i, k;
	struct Node *head, *newNode, *temp;
	printf("Enter number of nodes: ");
	scanf("%d", &n);
	head = NULL;
	for (i = 1; i <= n; i++)
	{
		newNode = (struct Node*)malloc(sizeof(struct Node));
		printf("Enter element %d: ", i);
		scanf("%d", &newNode->data);
		newNode->next = NULL;
		if (head == NULL)
		{
			head = newNode;
			temp = newNode;
		}
		else
		{
			temp->next = newNode;
			temp = newNode;
		}
	}
	displayList(head);
	printf("\n");
	printf("Enter number of nodes in group: ");
	scanf("%d", &k);
	head = reverseListInGroups(head, k);
	displayList(head);
	printf("\n");
   return 0;
}
struct Node *reverseListInGroups(struct Node *head, int k)
{
	int count;
	struct Node *ptr, *fast, *slow, *rev, *p, *q;
	ptr = head;
```

```
slow = fast = NULL;
count = k;
while (ptr != NULL && count != 0)
{
        count--;
        fast = ptr;
        ptr = ptr->next;
        fast->next = slow;
        slow = fast;
}
rev = fast;
while (ptr != NULL)
{
        p = rev;
        while (p->next != NULL)
        {
                p = p->next;
        }
        slow = fast = NULL;
        count = k;
        while (ptr != NULL && count != 0)
        {
                count--;
                fast = ptr;
                ptr = ptr->next;
                fast->next = slow;
                slow = fast;
        }
        q = fast;
        if (count != 0)
        {
                ptr = fast;
                slow = fast = NULL;
                while (ptr != NULL)
                {
                        fast = ptr;
                        ptr = ptr->next;
                        fast->next = slow;
                        slow = fast;
                }
                q = fast;
```

```
                }
                p->next = q;
        }
        return rev;
}
void displayList(struct Node *head)
{
        struct Node *ptr;
        ptr = head;
        while (ptr != NULL)
        {
                printf("%d\t", ptr->data);
                ptr = ptr->next;
        }
}
```

Output:

```
Enter number of nodes: 7
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Enter element 4: 4
Enter element 5: 5
Enter element 6: 6
Enter element 7: 7
1       2       3       4       5       6       7
Enter number of nodes in group: 3
3       2       1       6       5       4       7
```

6. **Check whether a singly linked list is a palindrome. Example: 9 → 8 → 8 → 9 → NULL → Palindrome**

```
// 6. Check whether a singly linked list is a palindrome.
#include <stdio.h>
#include <stdlib.h>
struct Node
{
  int data;
  struct Node *next;
};
struct Node *reverseList(struct Node *head);
int isPalindrome(struct Node *head);
```

```
int main()
{
    int n;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    struct Node *head, *newNode, *temp;
    head = NULL;
    temp = NULL;
    for (int i = 1; i <= n; i++)
    {
        newNode = (struct Node*)malloc(sizeof(struct Node));
        printf("Enter element %d: ", i);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
        if (head == NULL)
        {
            head = newNode;
            temp = newNode;
        }
        else
        {
            temp->next = newNode;
            temp = newNode;
        }
    }
    struct Node *ptr;
    ptr = head;
    while (ptr != NULL)
    {
        printf("%d\t", ptr->data);
        ptr = ptr->next;
    }
    if (isPalindrome(head))
    {
        printf("\nPalindrome\n");
    }
    else
    {
        printf("\nNot palindrome\n");
    }
    return 0;
```

```
}
struct Node *reverseList(struct Node *head)
{
   struct Node *p, *q, *ptr;
   ptr = head;
   q = p = NULL;
   while (ptr != NULL)
   {
      p = ptr;
      ptr = ptr->next;
      p->next = q;
      q = p;
   }
   head = p;
   return head;
}
int isPalindrome(struct Node *head)
{
   struct Node *q, *p;
   q = p = head;
   while (p != NULL && p->next != NULL)
   {
      q = q->next;
      p = p->next->next;
   }
   q = reverseList(q);
   p = head;
   while (q != NULL)
   {
      if (p->data != q->data)
      {
         return 0;
      }
      p = p->next;
      q = q->next;
   }
   return 1;
}
```

Output:

```
Enter number of nodes: 4
Enter element 1: 9
Enter element 2: 8
Enter element 3: 8
Enter element 4: 9
9       8       8       9
Palindrome
```

7. **Move the last node of a linked list to the front. Example: 10 → 20 → 30 → 40 → NULL → 40 → 10 → 20 → 30 → NULL**

```
// 7. Move the last node of a linked list to the front
#include <stdio.h>
#include <stdlib.h>
struct Node
{
  int data;
  struct Node *next;
};
struct Node *moveLastNodeToFront(struct Node *head);
int main()
{
  int n;
  printf("Enter number of nodes: ");
  scanf("%d", &n);
  struct Node *head, *newNode, *temp;
  head = NULL;
  for (int i = 1; i <= n; i++)
  {
    newNode = (struct Node*)malloc(sizeof(struct Node));
    printf("Enter element %d: ", i);
    scanf("%d", &newNode->data);
    newNode->next = NULL;
    if (head == NULL)
    {
      head = newNode;
      temp = newNode;
    }
    else
```

```
    {
      temp->next = newNode;
      temp = newNode;
    }
  }
  printf("Elements of linked list are ");
  struct Node *ptr;
  ptr = head;
  while (ptr != NULL)
  {
    printf("%d\t", ptr->data);
    ptr = ptr->next;
  }
  head = moveLastNodeToFront(head);
  printf("\nElements of linked list after moving last node to front are ");
  ptr = head;
  while (ptr != NULL)
  {
    printf("%d\t", ptr->data);
    ptr = ptr->next;
  }
  return 0;
}
struct Node *moveLastNodeToFront(struct Node *head)
{
  struct Node *fast, *slow;
  slow = fast = head;
  while (fast->next != NULL)
  {
    slow = fast;
    fast = fast->next;
  }
  slow->next = NULL;
  fast->next = head;
  head = fast;
  return head;
}
```

Output:

```
Enter number of nodes: 4
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30
Enter element 4: 40
Elements of linked list are 10  20      30      40
Elements of linked list after moving last node to front are 40  10      20 3
0
```

8. **Merge two sorted singly linked lists into one sorted list. Example: 1→4→8 and 2→3→9 → 1→2→3→4→8→9**

```
// 8. Merge two sorted linked lists into one sorted list
#include <iostream>
using namespace std;
struct Node {
  int data;
  Node *next;
};
void displayList(Node *head);
Node* mergeLists(Node *head1, Node *head2);
int main() {
  Node *head, *head1, *head2, *newNode, *temp;
  int n;
  head1 = NULL;
  cout << "Enter number of nodes: ";
  cin >> n;
  for (int i = 0; i < n; i++) {
    newNode = new Node;
    cout << "Enter element " << i + 1 << ": ";
    cin >> newNode->data;
    newNode->next = NULL;
    if (head1 == NULL) {
      head1 = newNode;
      temp = newNode;
    } else {
      temp->next = newNode;
      temp = newNode;
    }
  }
  head2 = NULL;
  cout << "Enter number of nodes: ";
  cin >> n;
  for (int i = 0; i < n; i++) {
    newNode = new Node;
```

```
        cout << "Enter element " << i + 1 << ": ";
        cin >> newNode->data;
        newNode->next = NULL;

        if (head2 == NULL) {
            head2 = newNode;
            temp = newNode;
        } else {
            temp->next = newNode;
            temp = newNode;
        }
    }
    displayList(head1);
    cout << endl;
    displayList(head2);
    cout << endl;
    head = mergeLists(head1, head2);
    displayList(head);
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
Node* mergeLists(Node *head1, Node *head2) {
    if (head1 == NULL) {
        return head2;
    } else if (head2 == NULL) {
        return head1;
    }
    Node *head, *ptr, *p, *q;
    p = head1;
    q = head2;
    if (p->data <= q->data) {
        ptr = p;
        p = p->next;
    } else {
```

```
        ptr = q;
        q = q->next;
    }
    ptr->next = NULL;
    head = ptr;
    while (p != NULL && q != NULL) {
        if (p->data <= q->data) {
            ptr->next = p;
            p = p->next;
        } else {
            ptr->next = q;
            q = q->next;
        }
        ptr = ptr->next;
    }
    if (p != NULL) {
        ptr->next = p;
    } else if (q != NULL) {
        ptr->next = q;
    }
    return head;
}
```

Output:

```
Enter number of nodes: 3
Enter element 1: 1
Enter element 2: 4
Enter element 3: 8
Enter number of nodes: 3
Enter element 1: 2
Enter element 2: 3
Enter element 3: 9
1       4       8
2       3       9
1       2       3       4       8       9
```

9. **Find the intersection point of two singly linked lists.**

// 9. Find the intersection point of two singly linked lists.

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node *next;
};
struct Node *findIntersectionPoint(struct Node *head1, struct Node *head2);
int main()
{
   struct Node *head1, *head2, *node1, *node2, *node3, *node4, *node5,
*node6, *node7;
   node1 = (struct Node*)malloc(sizeof(struct Node));
   node2 = (struct Node*)malloc(sizeof(struct Node));
   node3 = (struct Node*)malloc(sizeof(struct Node));
   node4 = (struct Node*)malloc(sizeof(struct Node));
   node5 = (struct Node*)malloc(sizeof(struct Node));
   node6 = (struct Node*)malloc(sizeof(struct Node));
   node7 = (struct Node*)malloc(sizeof(struct Node));
   head1 = node1;
   node1->data = 4;
   node1->next = node2;
   node2->data = 1;
   node2->next = node3;
   node3->data = 8;
   node3->next = node4;
   node4->data = 5;
   node4->next = NULL;
   head2 = node5;
   node5->data = 5;
   node5->next = node6;
   node6->data = 6;
   node6->next = node7;
   node7->data = 1;
   node7->next = node3;
   printf("The intersection point is the node whose data is %d\n",
findIntersectionPoint(head1, head2)->data);
   return 0;
}
struct Node *findIntersectionPoint(struct Node *head1, struct Node *head2)
{
```

```
  struct Node *p, *q;
  p = head1;
  while (p != NULL)
  {
    q = head2;
    while (q != NULL)
    {
      if (q == p)
      {
        return q;
      }
      q = q->next;
    }
    p = p->next;
  }
  return NULL;
}
```

Output:

```
The intersection point is the node whose data is 8
```

10. **Flatten a linked list having next and bottom pointers into a single sorted list.**

Output: 5 -> 7 -> 8 -> 10 -> 19 -> 20 -> 22 -> 28 -> 40 -> 45

```
// 10. Flatten a linked list having next and bottom pointers into a single sorted
list.
#include <stdio.h>
#include <stdlib.h>
struct Node
{
  int data;
  struct Node *next;
  struct Node *bottom;
};
struct Node *flattenList(struct Node *head);
struct Node *mergeLists(struct Node *head1, struct Node *head2);
int main()
{
  int i;
  struct Node *result, *head, *nodes[10];
  for (int i = 0; i < 10; i++)
  {
    nodes[i] = (struct Node*)malloc(sizeof(struct Node));
```

```
}

head = nodes[0];

nodes[0]->data = 5;
nodes[0]->next = nodes[1];
nodes[0]->bottom = nodes[4];

nodes[1]->data = 10;
nodes[1]->next = nodes[2];
nodes[1]->bottom = nodes[6];

nodes[2]->data = 19;
nodes[2]->next = nodes[3];
nodes[2]->bottom = nodes[7];

nodes[3]->data = 28;
nodes[3]->next = NULL;
nodes[3]->bottom = nodes[8];

nodes[4]->data = 7;
nodes[4]->next = NULL;
nodes[4]->bottom = nodes[5];

nodes[5]->data = 8;
nodes[5]->next = NULL;
nodes[5]->bottom = NULL;

nodes[6]->data = 20;
nodes[6]->next = NULL;
nodes[6]->bottom = NULL;

nodes[7]->data = 22;
nodes[7]->next = NULL;
nodes[7]->bottom = NULL;

nodes[8]->data = 40;
nodes[8]->next = NULL;
nodes[8]->bottom = nodes[9];

nodes[9]->data = 45;
```

```
    nodes[9]->next = NULL;
    nodes[9]->bottom = NULL;

    result = flattenList(head);
    struct Node *ptr;
    ptr = result;
    while (ptr != NULL)
    {
        printf("%d\t", ptr->data);
        ptr = ptr->bottom;
    }
    return 0;
}
struct Node *flattenList(struct Node *head)
{
    struct Node *ptr, *result;
    result = head;
    ptr = head->next;
    result->next = NULL;
    while (ptr != NULL)
    {
        struct Node *temp = ptr->next;
        ptr->next = NULL;
        result = mergeLists(result, ptr);
        ptr = temp;
    }
    return result;
}
struct Node *mergeLists(struct Node *head1, struct Node *head2)
{
    struct Node *head, *ptr, *p, *q;
    p = head1;
    q = head2;
    if (p->data <= q->data) {
        ptr = p;
        p = p->bottom;
    } else {
        ptr = q;
        q = q->bottom;
    }
    ptr->bottom = NULL;
```

```
    head = ptr;
    while (p != NULL && q != NULL) {
      if (p->data <= q->data) {
        ptr->bottom = p;
        p = p->bottom;
      } else {
        ptr->bottom = q;
        q = q->bottom;
      }
      ptr = ptr->bottom;
    }
    if (p != NULL) {
      ptr->bottom = p;
    } else if (q != NULL) {
      ptr->bottom = q;
    }
    return head;
  }
```

Output:

```
5        7        8        10       19       20       22       28       40       45
```

11. **Merge K sorted linked lists into one sorted linked list. Example: L1=1→5, L2=2→6, L3=3→4 → 1→2→3→4→5→6**

```
// 11. Merge k sorted linked lists into one sorted linked list
#include <iostream>
using namespace std;
struct Node {
  int data;
  Node *next;
};
void displayList(Node *head);
Node* mergeKLists(Node *heads[], int k);
Node* mergeLists(Node *head1, Node *head2);
int main() {
  int k, n;
  cout << "Enter k: ";
  cin >> k;
  Node *heads[k], *newNode, *temp, *head;
  for (int i = 0; i < k; i++) {
    heads[i] = NULL;
    cout << "Enter number of nodes: ";
    cin >> n;
```

```
        for (int j = 0; j < n; j++) {
            newNode = new Node;
            cout << "Enter element " << j + 1 << ": ";
            cin >> newNode->data;
            newNode->next = NULL;
            if (heads[i] == NULL) {
                heads[i] = newNode;
                temp = newNode;
            } else {
                temp->next = newNode;
                temp = newNode;
            }
        }
    }
    for (int i = 0; i < k; i++) {
        displayList(heads[i]);
        cout << endl;
    }
    head = mergeKLists(heads, k);
    displayList(head);
    cout << endl;
    return 0;
}
void displayList(Node *head) {
    Node *ptr;
    ptr = head;
    while (ptr != NULL) {
        cout << ptr->data << "\t";
        ptr = ptr->next;
    }
}
Node* mergeKLists(Node *heads[], int k) {
    if (k == 0) {
        return NULL;
    }
    for (int i = 0; i <= k - 2; i++) {
        heads[i + 1] = mergeLists(heads[i], heads[i + 1]);
    }
    return heads[k - 1];
}
Node* mergeLists(Node *head1, Node *head2) {
```

```
    if (head1 == NULL) {
        return head2;
    } else if (head2 == NULL) {
        return head1;
    }
    Node *head, *ptr, *p, *q;
    p = head1;
    q = head2;
    if (p->data <= q->data) {
        ptr = p;
        p = p->next;
    } else {
        ptr = q;
        q = q->next;
    }
    ptr->next = NULL;
    head = ptr;
    while (p != NULL && q != NULL) {
        if (p->data <= q->data) {
            ptr->next = p;
            p = p->next;
        } else {
            ptr->next = q;
            q = q->next;
        }
        ptr = ptr->next;
    }
    if (p != NULL) {
        ptr->next = p;
    } else if (q != NULL) {
        ptr->next = q;
    }
    return head;
}
```

Output:

```
Enter k: 3
Enter number of nodes: 2
Enter element 1: 1
Enter element 2: 5
Enter number of nodes: 2
Enter element 1: 2
Enter element 2: 6
Enter number of nodes: 2
Enter element 1: 3
Enter element 2: 4
1       5
2       6
3       4
1       2       3       4       5       6
```